
GIT
Node.js
TypeScript
Angular
React
Python
Matemáticas
Física
Ciencia de D
Inteligencia A
Arquitectura d
Blockchain
Autor
ZEDMOUS

ZEDMOUS.DEV

INGENIERÍA DE SOFTWARE, SISTEMAS DISTRIBUIDOS & CONTROL DE VERSIONES

GUÍA TÉCNICA OFICIAL: FLUJO DE TRABAJO PROFESIONAL CON GIT Y GITHUB

AUTORÍA & CÁTEDRA: ING. EDUARDO NIEVES (ZEDMOUS) • SERIE PROGRAMACIÓN (GIT-01) — EDICIÓN 2026

0. Mapa Estructural de Contenido y Arquitectura de Estados de Git

ÍNDICE GENERAL & FLUJO CORE

I. Introducción & Gestión Local

BLOQUES 01–02

1. Introducción:
- ¿Qué es Git?
- ¿Por qué debemos usarlo?
- ¿Cómo es el proceso de trabajo con Git?

2. Comandos con Gestión Local:
- `git init`, `git add`, `git status`
- `git commit`, `git log`

II. Repositorios & Ramas

BLOQUES 03–04

3. Gestión de Repositorios:
- `git clone`, `git remote`
- `git push`, `git pull`

4. Gestión de Ramas (Branches):
- `git branch` (`-a`, `-r`, `-D`)
- `git fetch`, `git checkout`, `git switch`, `git merge`

III. Conflictos, Prácticas & Cierre

BLOQUES 05–06

5. Conflictos y Casos Prácticos:
- Muestra y resolución de un conflicto (`UU`, `HEAD`)
- Prácticas: Examen de 20 preguntas sobre `Zedmous/zmgit`

6. Conclusiones:
- Síntesis profesional y buenas prácticas colaborativas.

1. Introducción: ¿Qué es Git, Por Qué Debemos Usarlo y Cómo es su Flujo de Trabajo?

FUNDAMENTOS DE CONTROL DE VERSIONES

¿Qué es Git? Es un **sistema de control de versiones distribuido** que permite gestionar los cambios en los archivos de un proyecto a lo largo del tiempo. Facilita el trabajo en equipo y la colaboración eficiente, garantizando que cada desarrollador posea un espejo íntegro del historial del repositorio.

1. TRABAJO EN EQUIPO & RED
Sincronización simultánea entre múltiples desarrolladores sin sobrescribir avances ajenos.

2. GRAFO DE RAMAS (BRANCHING)
Aislamiento seguro de nuevas funcionalidades mediante ramas paralelas (`main`, `dev`, `feature`).



ZEDMOUS.DEV · ARQUITECTO

3. CÓDIGO FUENTE (</>)
Auditoría línea por línea de cada modificación con huella criptográfica única `SHA-1`.

4. VERSIONADO V1 ↔ V2
Viaje en el tiempo instantáneo entre versiones estables del proyecto sin duplicar carpetas manualmente.

¿Por qué debemos usar Git? — Los 4 Pilares de la Ingeniería de Software Moderna

VENTAJAS COMPETITIVAS

- **1. CONTROL DE VERSIONES** Permite llevar un seguimiento exacto de todos los cambios realizados en el código, facilitando la identificación de errores y la restauración de versiones anteriores.
- **2. TRABAJO EN EQUIPO** Múltiples desarrolladores pueden trabajar en diferentes partes de un proyecto simultáneamente sin interferir en el trabajo de otros.
- **3. SEGURIDAD** Las versiones anteriores del proyecto quedan almacenadas en el historial, haciendo fácil revertir cualquier cambio no deseado o accidental.
- **4. RENDIMIENTO** Git trabaja de manera sumamente eficiente incluso con proyectos de gran escala gracias a su estructura distribuida local.

¿Cómo es el proceso de trabajo con Git? — Ciclo Operativo en 6 Etapas

WORKFLOW ESTÁNDAR

1. Clonar un repositorio: El primer paso suele ser copiar el repositorio desde un servidor remoto (como *GitHub*, *GitLab* o *Bitbucket*) a tu máquina local.

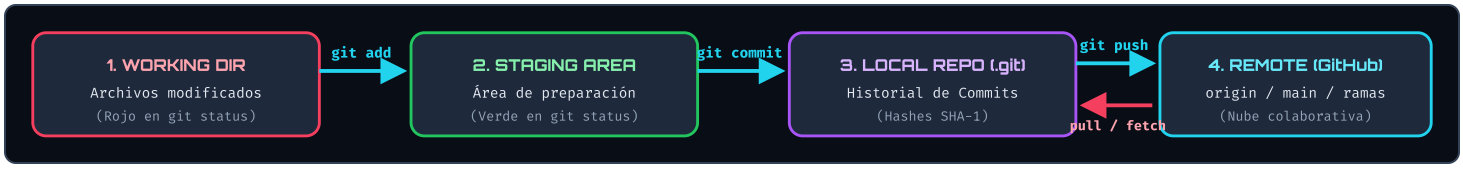
2. Crear ramas (branches): Los desarrolladores crean ramas para trabajar en nuevas características o soluciones a problemas de manera aislada, a partir de la rama principal (`main` o `master`).

3. Realizar cambios en los archivos: Una vez en la rama, se realizan las modificaciones necesarias en los archivos del proyecto.

4. Registrar cambios (commit): Los cambios realizados se registran en el historial del proyecto a través de un `"commit"`, el cual incluye un mensaje descriptivo sobre lo que se ha modificado.

5. Sincronizar con el repositorio remoto (push): Una vez registrados los cambios localmente, se suben al repositorio remoto para que el resto del equipo pueda acceder a ellos.

6. Integración de cambios (merge): Cuando el trabajo en una rama está completo y probado, se integra con la rama principal mediante un proceso llamado `"merge"` o a través de un `"pull request"`.



2. Comandos Básicos con Gestión Local: `git init`, `git add` y `git status`

REPOSITORIO LOCAL & STAGING AREA

`git init` Inicialización de un Nuevo Repositorio Local

CREACIÓN DE .git

Se utiliza para inicializar un nuevo repositorio de Git en un proyecto o carpeta local. Este comando crea un subdirectorio oculto llamado `.git`, donde se almacenan todos los datos y configuraciones necesarios para el control de versiones. Al ejecutarlo, el directorio actual comienza a ser rastreado por Git.

Este equipo > Disco local (C:) > dev > git > proyecto

Nombre	Fecha de modificación	Tipo
.git *(Carpeta oculta del sistema)*	07/06/2025 06:39 a. m.	Carpeta de archivos

MINGW64:/c/dev/git/proyecto

```
Zedmous@DESKTOP-OROG7AB MINGW64 /c/dev/git/proyecto
$ git init
Initialized empty Git repository in
C:/dev/git/proyecto/.git/

Zedmous@DESKTOP-OROG7AB MINGW64 /c/dev/git/proyecto (master)
$
```

`git add` Preparación de Archivos en el *Staging Area*

SINTAXIS INDIVIDUAL Y GLOBAL

Se utiliza para mover archivos al **"staging area"** (área de preparación). Esto indica a Git qué cambios específicos deseas incluir en el próximo `commit`, antes de que queden registrados oficialmente en el historial del proyecto.

1. Agregar un archivo específico:
Sintaxis: `git add <nombre-del-archivo>`
Ejemplo: `git add index.html`
Solo el archivo indicado se mueve al área de preparación.

2. Agregar todos los archivos modificados o nuevos:
Sintaxis: `git add .`
Incluye todos los archivos nuevos o modificados en el directorio actual y subdirectorios. ***Importante:*** *No incluye los listados en* `.gitignore`.

NOTA DE CONSOLA — ZEDMOUS:
Al crear `package.json` y ejecutar `git add .` en Windows, Git prepara el archivo para el próximo commit y normaliza automáticamente los saltos de línea (`LF` a `CRLF`).

Este equipo > Disco local (C:) > dev > git > proyecto

Nombre	Fecha de modificación	Tipo	Tamaño
.git	07/06/2025 06:39 a. m.	Carpeta de archivos	—
package.json	07/06/2025 06:46 a. m.	Archivo JSON	1 KB

MINGW64:/c/dev/git/proyecto (master)

```
Zedmous@DESKTOP-OROG7AB MINGW64 /c/dev/git/proyecto (master)
$ git add .
warning: in the working copy of 'package.json', LF will be replaced by CRLF the next time Git
touches it
```



`git status -s` (o `--short`) y Semántica de Colores en `git status`

INSPECCIÓN DEL ÁRBOL DE TRABAJO

Muestra el estado actual de los archivos en el repositorio pero en una **vista resumida y más compacta** que el comando tradicional `git status`. En lugar de descripciones largas, presenta una lista con dos columnas de símbolos para identificar rápidamente el estado de cada archivo:

Símbolo	Significado en `git status -s`
`M`	El archivo ha sido **modificado** respecto al último commit.
`A`	El archivo ha sido **añadido al área de preparación** (*staging area*).
`??`	El archivo **no está siendo rastreado** por Git (*untracked*).
`UU`	El archivo tiene un **conflicto de fusión (merge)**; ambos lados fueron modificados.

🟢 **Color VERDE en `git status`:** El archivo está en el **área de preparación** (*staging area*), listo para ser confirmado con `git commit` (ya ejecutaste `git add`).

🔴 **Color ROJO en `git status`:** El archivo ha sido modificado o no es rastreado, pero **aún no está en el área de preparación** (debes usar `git add` para incluirlo).



LECTURA RÁPIDA DE SÍMBOLOS Y COLORES:
Observa cómo `app.js` aparece con una **A en verde** (ya está en el *staging area* tras ejecutar `git add app.js`), mientras que `package.json` muestra **?? en rojo** porque aún no está siendo rastreado por Git.

C:\dev\git\proyecto

Nombre	Tipo	Tamaño
.git	Carpeta	—
app.js	Archivo JS	1 KB
package.json	Archivo JSON	1 KB

MINGW64:/c/dev/git/proyecto (master)

```
Zedmous@DESKTOP-OROG7AB MINGW64
/c/dev/git/proyecto (master)
$ git add app.js
$ git status -s
A  app.js
?? package.json
```

2.4 Confirmación de Cambios: `git commit -m` y `git commit -a -m`

SNAPSHOTS INMUTABLES SHA-1

`git commit -m "comentario"`

COMMIT ESTÁNDAR

Se utiliza para **registrar oficialmente los cambios** que han sido añadidos al área de preparación (*staging area*) en el historial del repositorio como un nuevo `commit`. El parámetro `-m` permite incluir un mensaje descriptivo directamente desde la línea de comandos, sin necesidad de abrir un editor externo.

MINGW64:/c/dev/git/proyecto (master)

```
Zedmous@DESKTOP-OROG7AB MINGW64 /c/dev/git/proyecto
(master)
$ git status -s
A  app.js
A  package.json

$ git commit -m "mi primer commit"
[master (root-commit) 27e6ada] mi primer commit
 2 files changed, 13 insertions(+)
 create mode 100644 app.js
 create mode 100644 package.json
```

`git commit -a -m "comentario"`

ATAJO TRACKED FILES

Realiza **dos acciones en una sola línea**:
1. Agrega automáticamente al *staging area* todos los archivos que **ya están siendo rastreados** por Git y han sido modificados o eliminados (*no incluye archivos nuevos; para esos sigue siendo necesario usar* ***`git add`***).
2. Crea el commit con el mensaje indicado tras `-m`.

MINGW64:/c/dev/git/proyecto (master)

```
Zedmous@DESKTOP-OROG7AB MINGW64 /c/dev/git/proyecto
(master)
$ git status -s
 M app.js

$ git commit -a -m "actualizacion del app"
[master f39fd8a] actualizacion del app
 1 file changed, 1 insertion(+), 1 deletion(-)
```

2.5 Auditoría del Historial de Versiones: `git log` y `git log --oneline`

TRAZABILIDAD CRONOLÓGICA

`git log` Historial Detallado de Commits (SHA-1 Completo, Autor y Fecha)

AUDITORÍA COMPLETA

Muestra el historial de commits de la rama activa. Incluye información detallada como el **identificador único (SHA-1)** de cada commit, el nombre y correo del autor, la fecha exacta y el mensaje descriptivo. Permite revisar los cambios realizados a lo largo del tiempo.

ANÁLISIS DE ZEDMOUS SOBRE `GIT LOG`:
«Acá tenemos más detalles de los dos commits realizados anteriormente en la práctica: cada bloque exhibe su firma criptográfica de 40 caracteres hexadecimales, el puntero `HEAD -> master`, el autor y la marca de tiempo exacta».

MINGW64:/c/dev/git/proyecto (master)

```
Zedmous@DESKTOP-OROG7AB MINGW64 /c/dev/git/proyecto (master)
$ git log
commit f39fd8a5500e40c6a76d891f8102a3855d8d51e6 (HEAD -> master)
Author: Zedmous <59229407+Zedmous@users.noreply.github.com>
Date:   Sat Jun 7 07:03:54 2025 -0400

    actualizacion del app

commit 27e6adaeb6ed40fd6f40bb610378d3bd0a3df3b4
Author: Zedmous <59229407+Zedmous@users.noreply.github.com>
Date:   Sat Jun 7 06:58:23 2025 -0400

    mi primer commit
```



ZEDMOUS · HISTORIAL

`git log --oneline` Vista Lineal y Compacta del Historial

RESUMEN DE 7 CARACTERES

Muestra una versión resumida del historial de commits, donde cada commit se presenta en **una sola línea**. Incluye únicamente los primeros 7 caracteres del identificador `SHA-1` y el mensaje del commit, ideal para inspeccionar rápidamente el progreso del proyecto.



ZEDMOUS · ONELINE

REGLA CRONOLÓGICA DE ZEDMOUS:
«Y acá se logra visualizar los dos commits de forma lineal y simple. Recordemos que la **primera línea** (arriba) es el **último commit realizado** (`f39fd8a`) y la **última línea** (abajo) es el **primer commit** (`27e6ada`)».

MINGW64:/c/dev/git/proyecto (master)

```
Zedmous@DESKTOP-OROG7AB MINGW64 /c/dev/git/proyecto (master)
$ git log --oneline
f39fd8a (HEAD -> master) actualizacion del app
27e6ada mi primer commit
```

3. Comandos Básicos Relacionados con Gestión de Repositorios Remotos

GITHUB, ORIGIN, PUSH & PULL

`git clone <url>` Clonación Íntegra de un Repositorio Remoto

COPIA DISTRIBUIDA

Se utiliza para copiar un repositorio existente (remoto o local) a tu máquina. Incluye todo su historial, ramas, commits y archivos. Este comando **descarga el contenido completo**, crea automáticamente la carpeta del proyecto y configura el repositorio remoto original bajo el alias predeterminado `origin`.

MINGW64:/c/dev/git

```
Zedmous@DESKTOP-OROG7AB MINGW64 /c/dev/git
$ git clone https://github.com/Zedmous/zmpython.git
Cloning into 'zmpython'...
remote: Enumerating objects: 19, done.
remote: Counting objects: 100% (19/19), done.
remote: Compressing objects: 100% (15/15), done.
remote: Total 19 (delta 6), reused 15 (delta 2), pack-reused 0 (from 0)
Receiving objects: 100% (19/19), 5.36 KiB | 1.79 MiB/s, done.
Resolving deltas: 100% (6/6), done.
```

`git remote` Vinculación, Actualización y Eliminación de Conexiones Remotas

GESTIÓN DE ORIGIN

1. Vincular un remoto nuevo (`git remote add origin <url>`): Conecta tu repositorio local con uno en la nube (GitHub, GitLab, etc.).

> **Desglose sintáctico:**
> - `git remote`: Gestiona las conexiones remotas.
> - `add`: Indica que vas a agregar una nueva conexión.
> - `origin`: Nombre estándar por convención para el remoto principal.
> - `<url>`: Dirección HTTPS o SSH del repositorio remoto.

2. Actualizar la URL (`git remote set-url origin <nueva-url>`): Reemplaza la URL actual cuando cambias el nombre del repositorio en GitHub, pasas de HTTPS a SSH o apuntas a otro servidor.

3. Eliminar el vínculo (`git remote remove origin`): Borra la referencia remota configurada en el proyecto local.

MINGW64:/c/dev/git/proyecto (master)

```
# 1. Vincular el repositorio local con GitHub:
Zedmous@DESKTOP-OROG7AB MINGW64 /c/dev/git/proyecto (master)
$ git remote add origin https://github.com/Zedmous/foc-git-2025-1.git

# 2. Actualizar la URL del remoto existente (set-url):
$ git remote set-url origin https://github.com/Zedmous/foc-git-2025-2.git

# 3. Eliminar la referencia remota si ya no se requiere:
$ git remote remove origin
```

`git push` / `git push -u origin <rama>`

PUBLICACIÓN DE COMMITS

Envía los commits realizados en tu repositorio local hacia el repositorio remoto (como `origin`). Actualiza la rama remota con los cambios confirmados localmente.

> **Importante (Vínculo Upstream `-u`):** Si es la **primera vez** que subes una rama, debes usar `git push -u origin <nombre-rama>` para vincular el seguimiento; luego bastará con escribir `git push`.

MINGW64:/c/dev/git/proyecto (master)

```
$ git push -u origin master
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (7/7), 618 bytes | 618.00 KiB/s,
done.
Total 7 (delta 0), reused 0 (delta 0), pack-reused 0 (from
0)
To https://github.com/Zedmous/foc-git-2025-1.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
```

`git pull` Sincronización y Fusión Remota

FETCH + MERGE

Actualiza tu repositorio local con los últimos cambios que existen en el repositorio remoto. Internamente realiza **dos operaciones seguidas**:

> **Ecuación Operativa de `git pull`:**
> `1. git fetch` (descarga los nuevos commits del remoto)
> `+ 2. git merge` (los fusiona con tu rama activa local).

MINGW64:/c/dev/git/proyecto (master)

```
$ git pull
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused
0 (from 0)
Unpacking objects: 100% (3/3), 958 bytes | 319.00 KiB/s,
done.
From https://github.com/Zedmous/foc-git-2025-1
   f39fd8a..cddbd8a  master     -> origin/master
Updating f39fd8a..cddbd8a
Fast-forward
 README.md | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
```

4. Comandos Básicos Relacionados con Ramas: `git branch`, `fetch`, `switch` y `merge`

DESARROLLO PARALELO AISLADO

`git branch`, `git branch -a` y `git branch -r` — Inspección de Ramas Locales y Remotas

LISTADO Y PERSPECTIVAS

El comando `git branch` permite visualizar y gestionar las ramas dentro de un repositorio. Al ejecutarlo sin argumentos, muestra la lista de ramas locales e indica con un **asterisco (`*`) en color verde** cuál es la rama activa en la que te encuentras situado.

EXPLICACIÓN DE ZEDMOUS SOBRE RAMAS LOCALES VS REMOTAS:

«Acá se visualizan las ramas existentes de forma local (`local` y `* master`). Al usar `git branch -a` vemos todas las ramas (locales y remotas en rojo), y con `git branch -r` solo las remotas. **¡Ojo!** En el ejemplo existen realmente **2 ramas en total** (`local` y `master`), solo que `master` está tanto en tu máquina como en `remotes/origin/master`».

git branch (Locales)

```
$ git branch
  local
* master
```

git branch -a / -r (Todas y Remotas)

```
$ git branch -a
  local
* master
  remotes/origin/master
$ git branch -r
  origin/master
```



Crear y Eliminar Ramas (`branch` / `branch -D`)

CICLO DE VIDA DE RAMAS

- **Crear rama (`git branch <nombre-rama>`):** Crea una nueva rama a partir del punto actual del historial sin cambiar automáticamente a ella.
- **Eliminar rama (`git branch -D <nombre-rama>`):** Elimina de forma forzada una rama local aunque tenga cambios no fusionados. *Importante: debes estar ubicado en una rama diferente a la que deseas eliminar.*

MINGW64:/c/dev/git/proyecto (master)

```
$ git branch crear-rama
$ git branch
  crear-rama
  local
* master

$ git branch -D crear-rama
Deleted branch crear-rama (was cddbd8a).
```

Navegación entre Ramas (`checkout` y `switch`)

CAMBIO DE PUNTERO HEAD

Para desplazarte hacia otra rama y activar sus archivos en tu directorio de trabajo, Git ofrece el comando clásico y el comando moderno especializado:

- `git checkout <nombre-rama>`: Cambia a la rama indicada (con `git checkout -b <rama>` la crea y salta a ella en un solo paso).
- `git switch <nombre-rama>`: Comando moderno dedicado exclusivamente a cambiar de rama (con `git switch -c <rama>` la crea y activa).

MINGW64:/c/dev/git/proyecto

```
$ git checkout local
Switched to branch 'local'
$ git switch master
Switched to branch 'master'
```

`git fetch` y `git fetch --all`

DESCARGA SIN FUSIÓN AUTOMÁTICA

`git fetch`: Descarga los últimos cambios del repositorio remoto (nuevos commits, ramas o etiquetas), pero **sin fusionarlos automáticamente** con tu rama local. Actualiza las referencias remotas como `origin/main` u `origin/dev` para que puedas revisar qué se ha hecho antes de decidir integrarlo.

`git fetch --all`: Ejecuta el proceso de actualización para **todos los repositorios remotos** configurados en tu proyecto (por ejemplo, tanto `origin` como `upstream`).

`git merge <nombre-rama>`

INTEGRACIÓN DE HISTORIALES

Combina los cambios de otra rama con la rama actual en la que te encuentras situado. Tiene **dos resultados posibles**:

1. Fusión exitosa: Los cambios se integran automáticamente sin problemas (*Fast-forward* o *Merge commit* limpio).

2. Fusión con conflictos: Si ambas ramas modificaron las mismas líneas de un archivo, Git detiene el proceso y marca los conflictos para que los resuelvas manualmente y confirmes con `git add` + `git commit -m`.

5. Conflictos y Casos Prácticos: Muestra y Resolución Paso a Paso de un Conflicto

RESOLUCIÓN DE MERGE CONFLICTS

Anatomía en VS Code de un Conflicto (Estado `UU`)

MARCADORES HEAD VS RAMA

app.js — proyecto (Merge Conflict) **UU (Ambos modificados)**

Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes

```
<<<<<<< HEAD (Current Change)
console.log("Hola mundo desde la rama master");
=======
console.log("Hola mundo desde la rama local");
>>>>>>> local (Incoming Change)
```

Protocolo Oficial de 4 Pasos para Resolver el Conflicto

FLUJO DE SOLUCIÓN

PASO 1 **Abrir el archivo en conflicto:** Detectado con `git status -s` (símbolo `UU`). Verás bloques delimitados por `<<<<<<< HEAD`, `=======` y `>>>>>>>`.

PASO 2 **Identificar y resolver los cambios:** Entre `<<<<<<< HEAD` y `=======` están los cambios de tu rama actual; debajo de `=======` hasta `>>>>>>>` están los de la otra rama. Decide cuál conservar o combínalos.

PASO 3 **Guardar la versión final:** Elimina las líneas de conflicto (`<<<<<<<`, `=======`, `>>>>>>>`) y guarda el archivo.

PASO 4 **Confirmar en Git:** Ejecuta `git add .` (o `git add <archivo>`) y cierra la fusión con `git commit -m "comentario"`.

5.2 Prácticas: Examen Práctico de Git y Control de Versiones (Repositorio Oficial `Zedmous/zmgit`)

LABORATORIO EVALUADO • 20 PREGUNTAS

Repositorio Base Oficial: `https://github.com/Zedmous/zmgit` • **Objetivo:** Evaluar el manejo práctico de comandos Git, comprensión de ramas, historial de cambios, diferencias entre versiones y análisis de estructura del proyecto.
Instrucciones Generales: 1. No descargues el proyecto en ZIP; usa exclusivamente comandos de Git desde la terminal. **2.** Todas las respuestas deben estar justificadas con base en los resultados obtenidos mediante comandos Git.

01. Clona el repositorio en tu máquina local (no lo descargues en ZIP).

02. ¿Cuántas ramas existen en el repositorio al clonarlo? Indica sus nombres.

03. ¿Cuántos commits tiene cada una de las ramas?

04. ¿Cuál es el primer y último commit de la rama `master` (muestra su mensaje y hash)? Para las demás ramas, indica solo el último commit.

05. Crea una nueva rama con las iniciales de tu nombre a partir de la rama `master`, y luego fusiona en ella las ramas restantes.

06. ¿Qué comando utilizaste para listar todas las ramas locales y remotas? ¿Qué diferencias notaste entre `git branch`, `git branch -a` y `git branch -r`?

07. ¿Qué comando te permite ver el historial de commits de forma resumida?

08. ¿Cuál es el mensaje del commit más reciente de la rama `rama-a`?

09. ¿Qué comando usaste para cambiar de una rama a otra?

10. Explica cómo se resolvió el conflicto si ocurrió durante alguna fusión.

11. ¿Cómo puedes visualizar los cambios pendientes de confirmar en tu rama actual?

12. ¿Qué comando utilizaste para crear tu nueva rama a partir de `master`?

13. ¿Qué significa el prefijo `origin/` que aparece en algunas ramas?

14. ¿Cuál fue el resultado final del `git merge` en tu rama personalizada?

15. ¿Qué archivos cambiaron después de cada fusión? Usa `git status` y `git diff` para justificar.

16. ¿Qué comando permite ver gráficamente el historial de ramas y fusiones?

17. ¿Qué sucede si eliminas una rama que estás usando? ¿Cómo evitaste este error?

18. ¿Cómo identificas un conflicto en Git y qué símbolos lo indican dentro del archivo?

19. ¿Qué comandos usaste para confirmar la resolución de un conflicto y finalizar la fusión?

20. ¿Qué aprendiste sobre el flujo de trabajo con ramas y fusiones a partir de esta práctica?

6. Conclusiones

SÍNTESIS PROFESIONAL ZEDMOUS.DEV

• El dominio de **Git** es fundamental en el desarrollo moderno de software, ya que permite llevar un control preciso de los cambios, colaborar eficientemente en equipo y mantener la integridad del código a lo largo del tiempo.

• Comprender comandos básicos como `init`, `add`, `commit`, `status`, `log`, así como el manejo de ramas y resolución de conflictos, brinda una base sólida para trabajar en entornos reales de desarrollo.



ZEDMOUS.DEV · CIERRE

• Además, saber interactuar con repositorios remotos mediante `clone`, `push`, `pull` y `fetch` fortalece la capacidad de integrarse en flujos de trabajo colaborativos y distribuidos.

ECOSISTEMA GIT + GITHUB
github.com/Zedmous • zedmous.netlify.app

AUTORÍA, DISEÑO & DOCENCIA:
Ing. Eduardo Nieves (Zedmous)
zedmous.netlify.app